Lab 8 — Docker, Containers and Images

Batoul Hachem | AUB username: bah38

Software Tools Laboratory | 9 November 2026

Introduction

A Docker image is a packaged application containing its code, runtime, libraries and configuration. A container is an instance of an image. The Dockerfile defines how to build the image. This lab builds a Flask hello-world image, runs the original Lab 5 API on port 5000, and adds a database-backed names API on port 3000.

Lab 5 source: https://github.com/bah38/Lab5_Postman_API (commit 85237839a037ce790bfec7f9f2ff5c1b525e1e5b). The repository database initially contains no users. The original files were preserved; working copies were used for containerization.

Introductory Flask container

The app returns Hello, World! at GET /. Flask listens on 0.0.0.0:5000 so it accepts Docker-forwarded traffic. The Dockerfile copies requirements.txt, installs Flask, copies app.py and runs python app.py. Python 3.12 Alpine is used as a maintained replacement for the handout's Python 3.8 example. A focused requirements file avoids copying unrelated local packages into the image.

```
docker build -t flask_docker ./hello_world
docker run --name lab8-bah38-hello -p 127.0.0.1:5000:5000 -d flask_docker
curl.exe http://localhost:5000/
```

Activity 1 — Commands and explanations

The handout's <image_name> placeholders for stop and logs refer to a container name or ID. The container used below is named lab8-bah38-hello; its image is flask_docker.

1. List all containers

```
docker ps -a
```

Lists running and stopped containers, including their IDs, images, commands, status, port mappings and names. The -a flag includes containers that are no longer running.

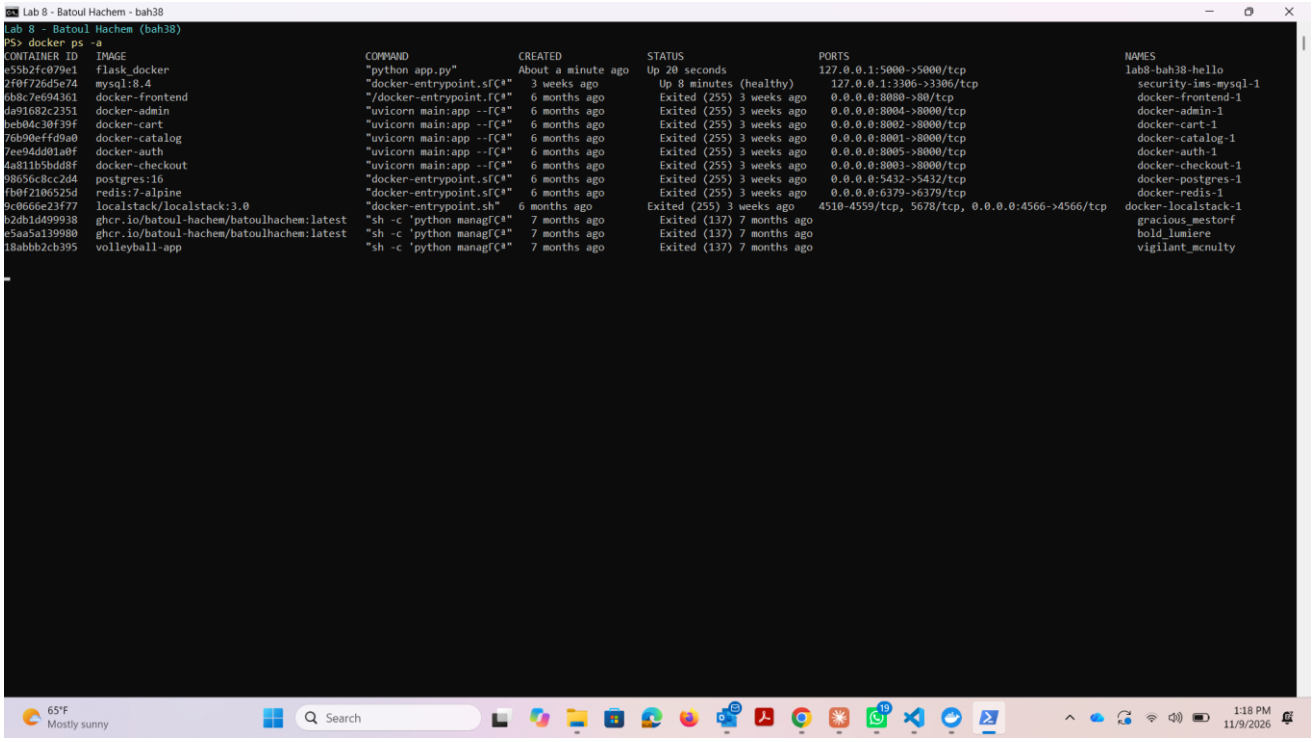


Figure 1. Actual output of docker ps -a.

2. Stop the Flask container

`docker stop lab8-bah38-hello`

Requests a graceful shutdown of the running container. Docker prints its name when it stops. The container remains available and can be restarted; stopping it does not delete its image or filesystem.

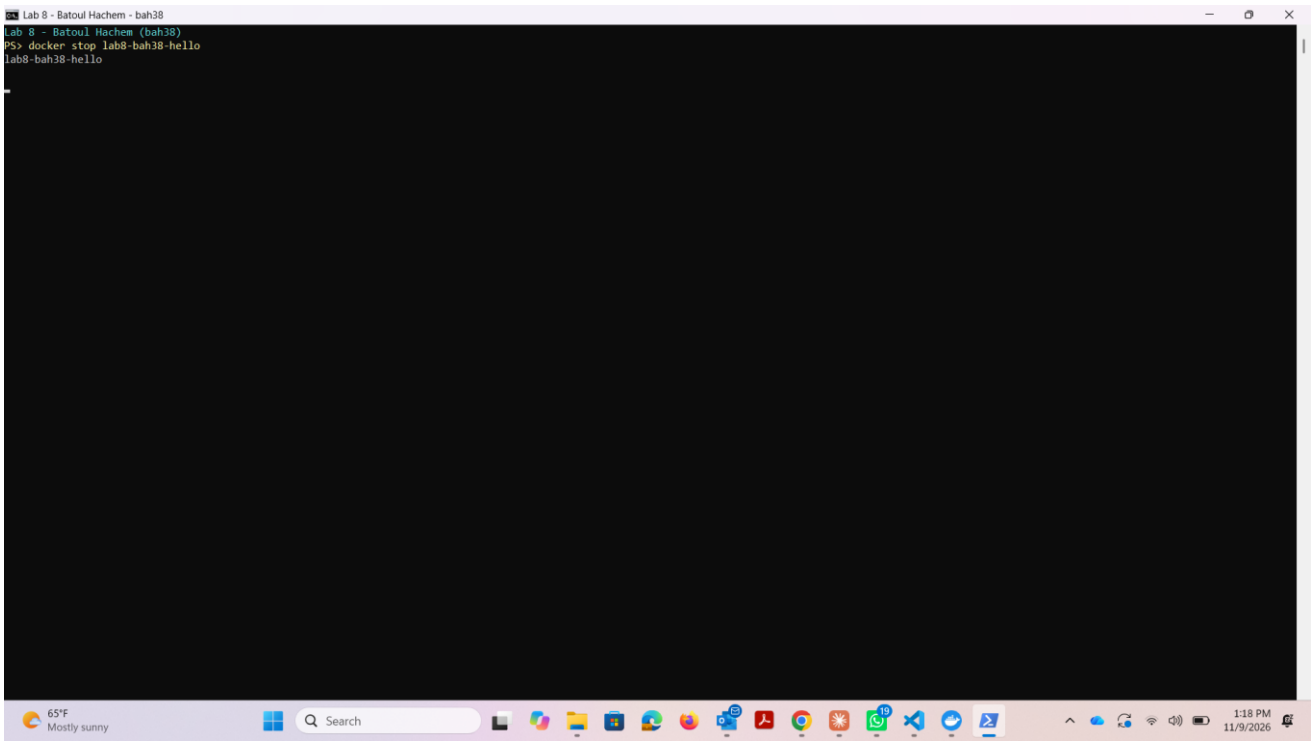


Figure 2. Actual output of docker stop.

3. View the container logs

`docker logs lab8-bah38-hello`

Displays the container's captured standard output and standard error. Flask startup and HTTP request messages appear here. Logs can still be read after the container has stopped.

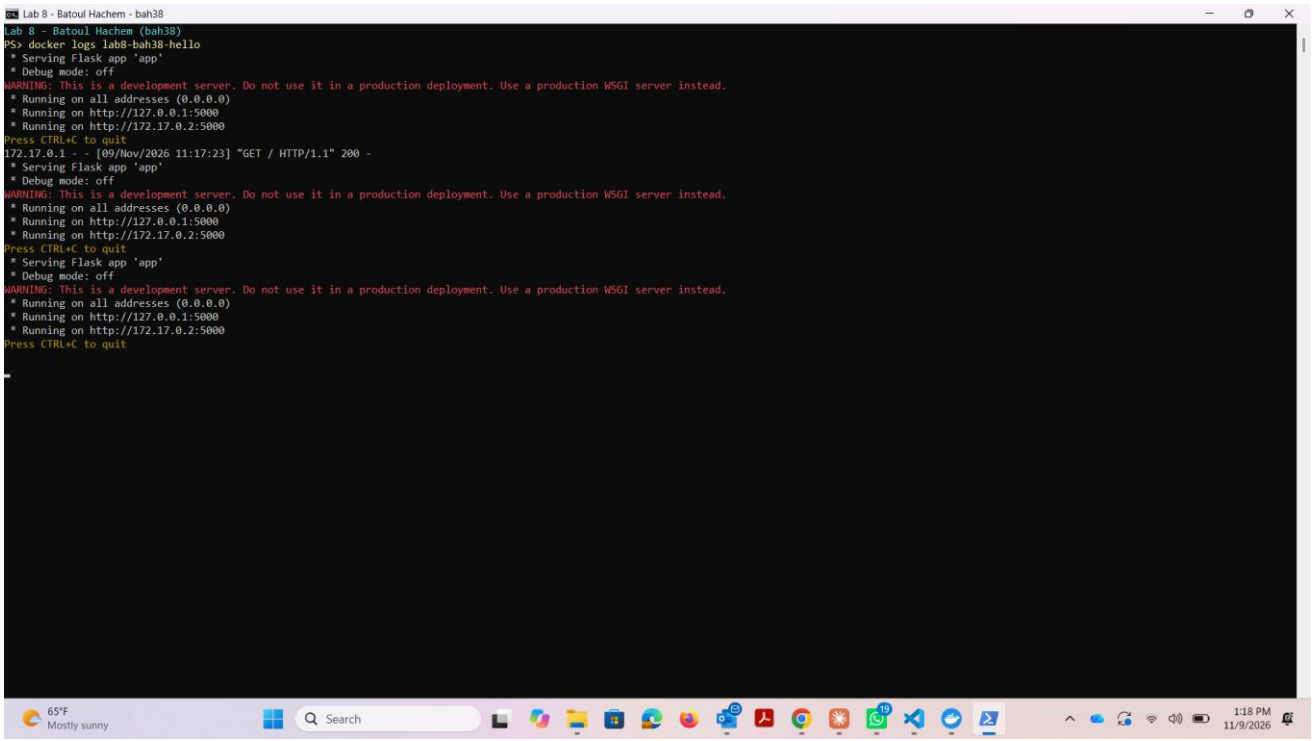


Figure 3. Actual Flask container logs after stopping.

Confirm stopped state

In this run, Exited (137) indicates that Docker forcefully terminated the development server after its stop grace period. The container is still retained and its logs remain readable.

`docker ps -a --filter name=lab8-bah38-hello`

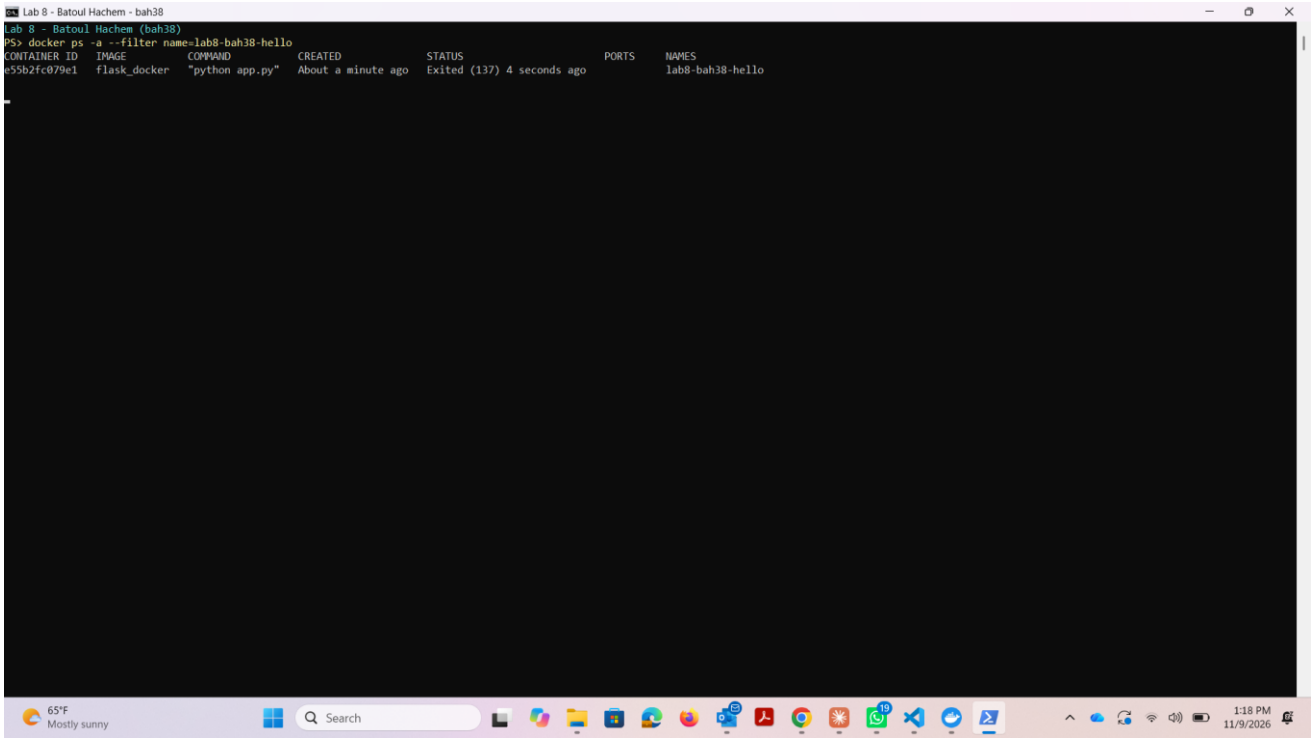


Figure 4. The hello-world container remains listed with an Exited status.

Activity 2 — Two application images

Image	Port	Application
bah38/lab8-batoul_hachem:0.0.1.RELEASE	5000	Original Lab 5 Postman/API application
bah38/lab8-batoul_hachem:0.0.2.RELEASE	3000	GET /get_users returns every user's name

The handout's embedded Docker Desktop screenshot uses tags 0.0.1.RELEASE and 0.0.2.RELEASE. These valid tags are used here because the textual 0.0.1:RELEASE and 0.0.2:RELEASE forms contain an extra colon. Repository names use lowercase letters.

Image 1: Lab 5 API on port 5000

The original Lab 5 application and database are copied into lab5_api/. The application retains its existing GET, POST, PUT and DELETE routes. Its only application-code change is app.run(host="0.0.0.0", port=5000, debug=False), allowing requests through the published container port. Flask and Flask-CORS are installed from requirements.txt.

```
docker build -t bah38/lab8-batoul_hachem:0.0.1.RELEASE ./lab5_api
docker run --name lab8-bah38-api5000 -p 127.0.0.1:5000:5000 -d bah38/lab8-batoul_hachem:0.0.1.RELEASE
```

The introductory container is stopped before running this container because both use host port 5000.

Comparison with the Lab 5 submission PDF

The supplied lab5_repo+screenshots (2).pdf identifies the same GitHub repository used for this image. Its five Postman examples match the original application's routes and the successful container verification below. Page 1 shows an empty users list ([]); pages 2–4 show a temporary user being created, read and updated; page 5 shows

successful deletion. The repository's empty database is consistent with these examples. No API changes were required after this comparison.

PDF page	Request	Comparison
1	GET /api/users	Same route; empty list returns 200
2	POST /api/users/add	Same route and six response fields
3	GET /api/users/<user_id>	Same route and user object
4	PUT /api/users/update	Same route and updated user object
5	DELETE /api/users/delete/<user_id>	Same route and success message

Image 2: Names API on port 3000

GET /get_users executes SELECT name FROM users ORDER BY user_id against a copy of the Lab 5 SQLite database and returns a JSON list of names. The result comes from the database on each request. An empty database returns []. SQLite connections are closed after use. Each image has its own database copy; changes in one container do not automatically appear in the other.

```
docker build -t bah38/lab8-batoul_hachem:0.0.2.RELEASE ./users_api
docker run --name lab8-bah38-api3000 -p 127.0.0.1:3000:3000 -d bah38/lab8-batoul_hachem:0.0.2.RELEASE
```

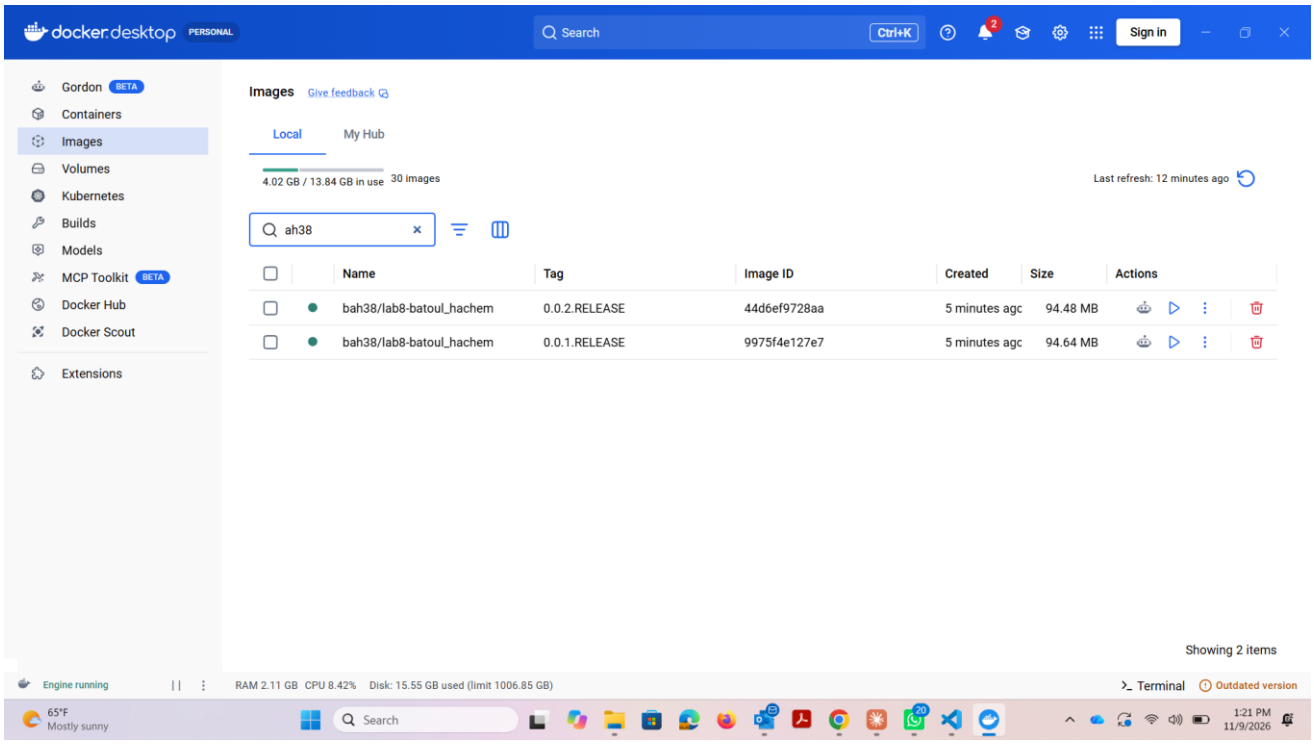


Figure 5. Docker Desktop Images view showing both required image tags.

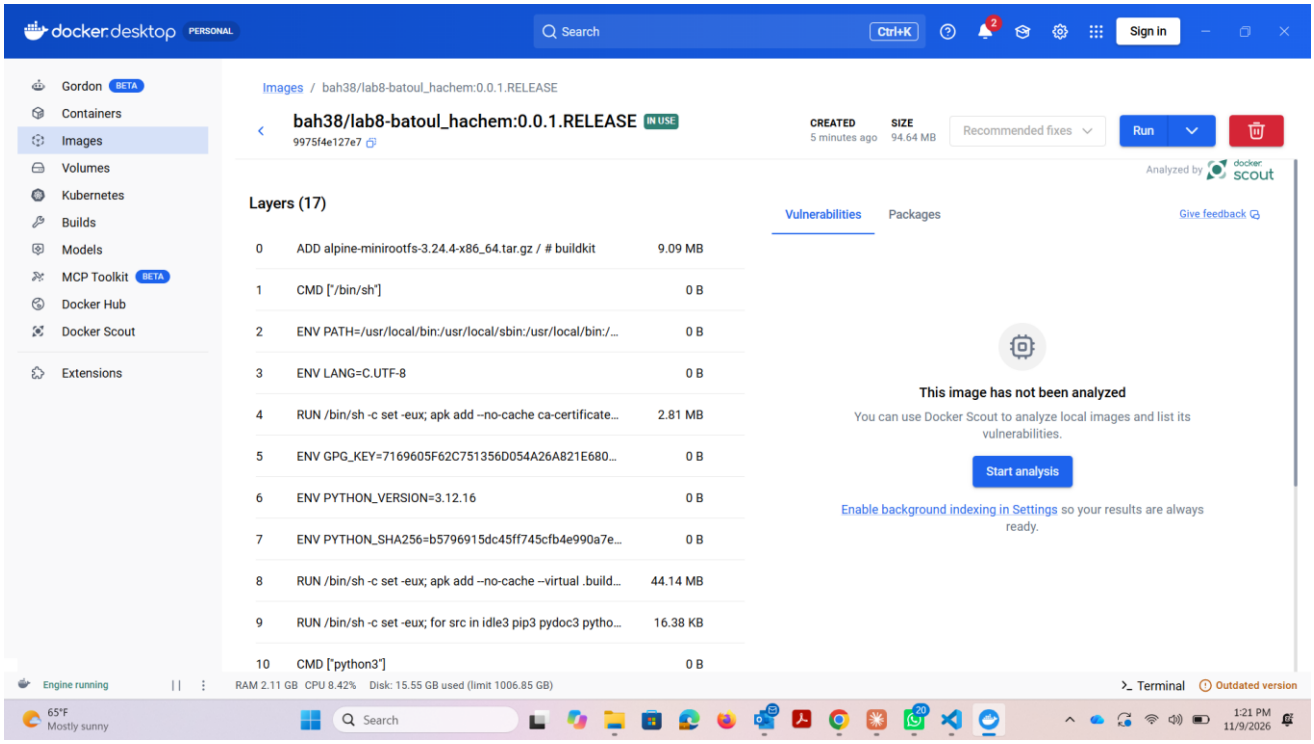


Figure 6. Docker Desktop image details for the port-5000 application.

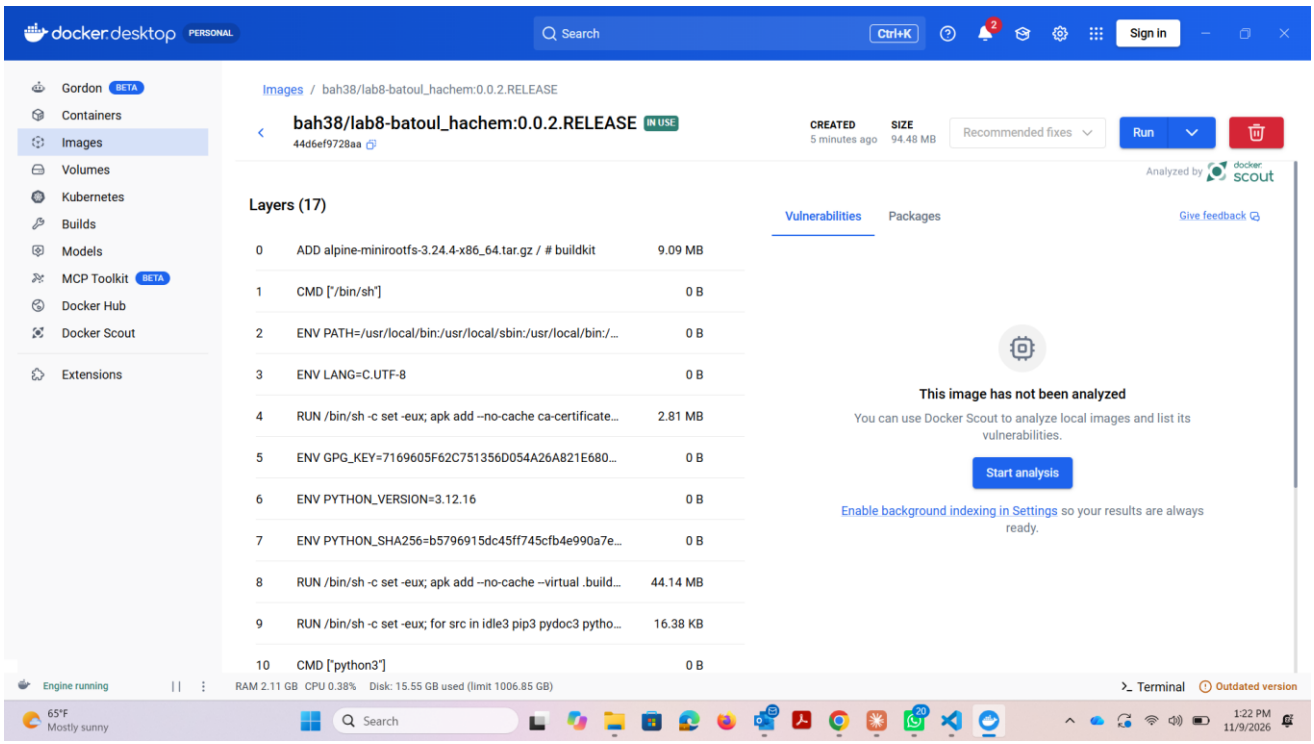


Figure 7. Docker Desktop image details for the port-3000 application.

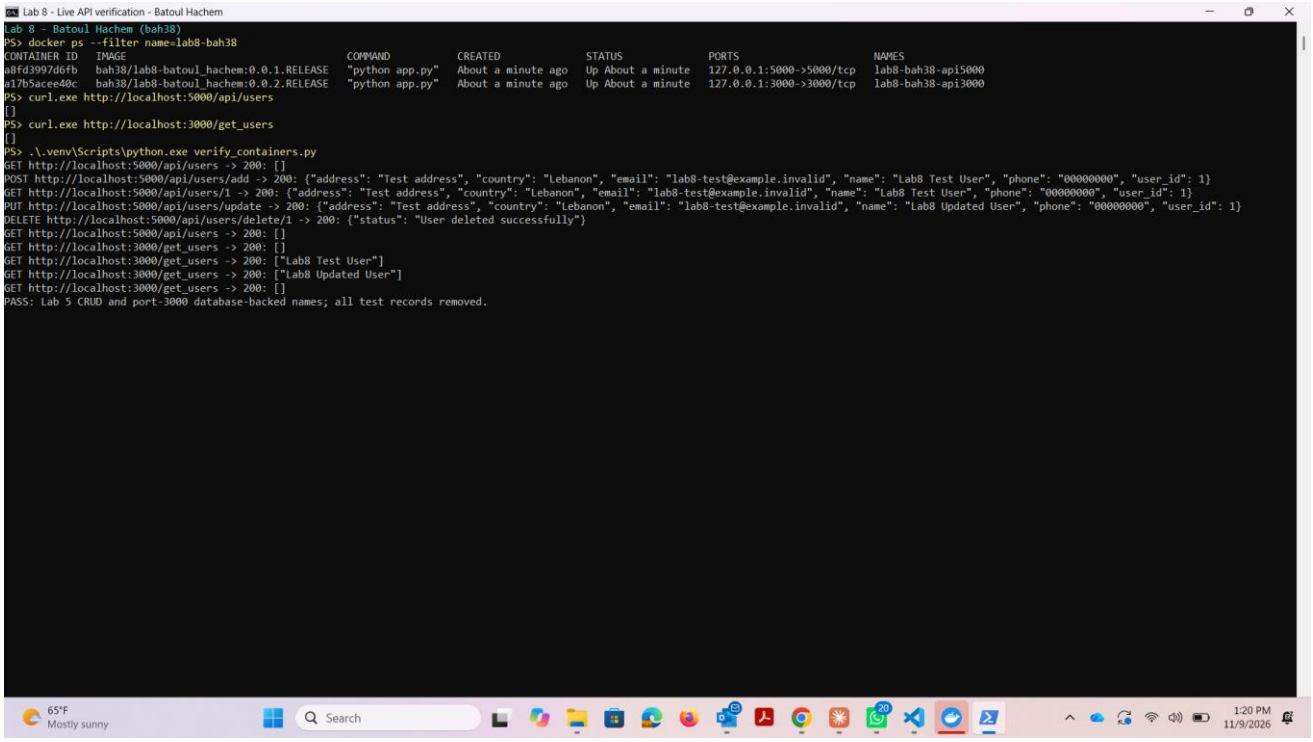


Figure 8. Live API requests to the two application containers.

Verification

Application checks cover the hello-world response, an empty users database, multiple user names, an updated record and rejection of POST to /get_users. Container verification performs the original Lab 5 create/read/update/delete workflow and checks that inserting and updating a temporary database record changes the names returned on port 3000. Temporary records are removed afterward; the bundled database files remain unchanged.

```
GET http://localhost:5000/api/users -> 200: []
POST http://localhost:5000/api/users/add -> 200: {"address": "Test address", "country": "Lebanon",
"email": "lab8-test@example.invalid", "name": "Lab8 Test User", "phone": "00000000", "user_id": 1}
GET http://localhost:5000/api/users/1 -> 200: {"address": "Test address", "country": "Lebanon",
"email": "lab8-test@example.invalid", "name": "Lab8 Test User", "phone": "00000000", "user_id": 1}
PUT http://localhost:5000/api/users/update -> 200: {"address": "Test address", "country": "Lebanon",
"email": "lab8-test@example.invalid", "name": "Lab8 Updated User", "phone": "00000000", "user_id": 1}
DELETE http://localhost:5000/api/users/delete/1 -> 200: {"status": "User deleted successfully"}
GET http://localhost:5000/api/users -> 200: []
GET http://localhost:3000/get_users -> 200: []
GET http://localhost:3000/get_users -> 200: ["Lab8 Test User"]
GET http://localhost:3000/get_users -> 200: ["Lab8 Updated User"]
GET http://localhost:3000/get_users -> 200: []
PASS: Lab 5 CRUD and port-3000 database-backed names; all test records removed.
```

References

Original Lab 5 repository: https://github.com/bah38/Lab5_Postman_API

User-supplied Lab 5 reference: lab5_repo+screenshots (2).pdf, pages 1–5.

Docker stop: https://docs.docker.com/reference/cli/docker/container/stop/

Docker image references: https://docs.docker.com/reference/cli/docker/image/tag/

Flask quickstart: https://flask.palletsprojects.com/en/stable/quickstart/